Lab 2 – Smart City SRS

Nick Kooshki

Old Dominion University

CS 411W

Dr. Sumaya Sanober

28 March 2026

Version 2

Table of Contents

1 Introduction 3

1.1 Purpose 3

1.2 Scope 4

1.3 Definitions, Acronyms, and Abbreviations 4

1.4 References 5

1.5 Overview 6

2 Overall Description 6

2.1 Product Perspective 6

2.2 Product Functions 7

2.3 User Characteristics 8

2.4 Constraints 9

2.5 Assumptions and Dependencies 9

List of Figures

Figure 1: Smart City Major Functional Component Diagram 7

1 Introduction

Cities are getting bigger and busier every year. The United Nations says that by 2050, around 6.5 billion people will be living in cities. Because of this, people deal with more traffic, less space, and slower communication between city systems. The way most cities are built cannot handle this kind of growth, so things like traffic, pollution, and poor resource management keep getting worse.

Smart City is our solution to this problem. It is a community-based app that helps people get around town faster and stay informed. The app uses AI and real-time data from users to make smarter route suggestions and share useful updates. The main goal is to reduce congestion and make it easier for residents, commuters, and city officials to stay connected. Smart City helps people and technology work together so cities can be more organized, efficient, and enjoyable to live in.

1.1 Purpose

Smart City is a mobile application designed for local city residents and commuters. The application provides an interactive interface optimized specifically for iOS and Android smartphones.

Key features for residents include Account Management, where users can securely log in, create, or delete their accounts. The Basic Navigation System allows users to view maps and routing info to navigate the city. The Community Forum allows residents to view and create posts mapped to specific locations, enabling communication with other community members. The application also includes an Alerts and Notifications feature to keep users informed about important posts. To ensure privacy, the Data Anonymity feature protects users' personal data while utilizing the platform.

Smart City is not a fully dynamic traffic control tool. Users are able to navigate the city, but the prototype will not feature the complex Dynamic Queue System for real-time traffic load balancing. Nor is Smart City intended to be a data analytics platform for local governments; the prototype will not include the dashboards, insights, and visualizations originally envisioned for

city officials. The aim is to establish a secure, developer-ready foundation for a community-based navigation application.

1.2 Scope

The goal of Smart City is to build a community-based navigation platform that mitigates urban congestion by combining user data with geospatial mapping. The prototype will provide a functional cross-platform mobile application where users can create secure accounts, navigate via an integrated map interface, and communicate with other residents through a localized community forum.

The prototype will serve as an initial case study deployed to a controlled user base of Old Dominion University (ODU) commuters. During this phase, the prototype will specifically test core features including simulated traffic alerts, community event notifications, and localized safety updates. We will collect quantitative data on routing efficiency and qualitative user feedback regarding the responsiveness of the alternative destination system and the clarity of the navigation UI. Analyzing these results will allow the development team to refine the routing logic, patch UI bugs, and enhance system stability before expanding the platform to the broader Hampton Roads population.

Due to the limited development timeline, the prototype will not implement the fully envisioned Dynamic Queue System for intelligent, real-time rerouting, utilizing basic A-to-B navigation instead. The system will also not include the advanced data analytics and visualization dashboards intended for city officials, simulating crowd-level alerts to demonstrate proof-of-concept rather than utilizing live data streams.

1.3 Definitions, Acronyms, and Abbreviations

API (Application Programming Interface): A set of protocols allowing different software components to communicate (e.g., REST APIs).

Coder Workspace: The containerized development environment hosted on the Old Dominion University (ODU) Computer Science Virtual Machine.

ElysiaJS: A fast server framework for JavaScript/TypeScript used to build the backend Application Server.

MFCD: Major Functional Component Diagram.

MySQL: An open-source relational database management system used for the Data Layer.

Privacy-by-Design: Building systems so that, by default, only the personal data strictly necessary for a purpose is processed and anonymized.

React Native: An open-source UI software framework used to develop the cross-platform (iOS/Android) mobile application.

RWP: Real-World Product.

UI: User Interface.

1.4 References

Gorzelany, J. (2024, August 30). Here's where commuters lose the most time and money sitting in traffic, study shows. Forbes. https://www.forbes.com/sites/jimgorzelany/2024/08/30/heres-where-commuters-lose-the-most-time-and-money-sitting-in-traffic-study-shows/

Joyce, H. (2025, March 26). Urban overcrowding: A major driver of pollution crises. ShunWaste. https://shunwaste.com/article/how-is-overcrowding-in-cities-causing-more-pollution

King, G. (n.d.). Data sharing: The next step in municipal collaboration. Data-Command. https://www.data-command.com/the-cloud/data-sharing-the-next-step-in-municipal-collaboration

Lee, S. (2025, May 27). Understanding urban overcrowding: The social and economic implications of overcrowding in cities. Number Analytics. https://www.numberanalytics.com/blog/ultimate-guide-to-overcrowding-in-urban-sociology

Luoma, K. (2016, November 21). How does population growth impact rapid urbanization? Population Education. https://populationeducation.org/how-does-population-growth-impact-rapid-urbanization/

Poindexter, D. (2018, January 24). The effects of overcrowding and the behavioral sink theory. The Borgen Project. https://borgenproject.org/theory-behavioral-sink/

Smith, H. (2021, June 18). Post-COVID L.A. traffic analysis: Has rush hour changed? Los Angeles Times. https://www.latimes.com/california/story/2021-06-18/post-covid-l-a-traffic-analysis-has-rush-hour-changed

Weinberg, N. (2016, August 11). How can cities be preemptive and effective in preventing overcrowding? Data-Smart City Solutions. https://datasmart.hks.harvard.edu/news/article/how-can-cities-be-preemptive-and-effective-in-preventing-overcrowding-888

Zurich Insurance Group. (2023, April 13). The risks of rapid urbanization in developing countries. https://www.zurich.com/knowledge/topics/global-risks/the-risks-of-rapid-urbanization-in-developing-countries

1.5 Overview

The remainder of this document outlines the overall technical description of the Smart City prototype. Section 2 details the product perspective, architectural subsystems, primary software functions, and user characteristics. It also defines the technical constraints, assumptions, and third-party dependencies that developers must adhere to during implementation.

2 Overall Description

2.1 Product Perspective

The Smart City prototype operates as an independent cross-platform mobile application that relies on external REST APIs for geospatial data. Structurally, the system is divided into three interacting subsystems **(see Figure 1)**:

1. **Presentation Layer (User Application)**: Built using the React Native framework for iOS and Android. This layer handles the graphical user interface, including user login screens, community forum interfaces, and interactive map controls rendered via the react-native-maps library.

2. **Application Layer (Application Server)**: Contains the central backend utilizing the ElysiaJS framework. It is responsible for user session management, core business logic, geospatial querying algorithms, and routing. It acts as the intermediary, securely passing REST API calls between the frontend and the database, and fetching mapping data from Mapbox and OpenStreetMap.
3. **Data Layer**: Consists of a MySQL Relational Database hosted on the Old Dominion University Computer Science Virtual Machine via Coder Workspace. It stores User Data, Forum Data, and Location Data.

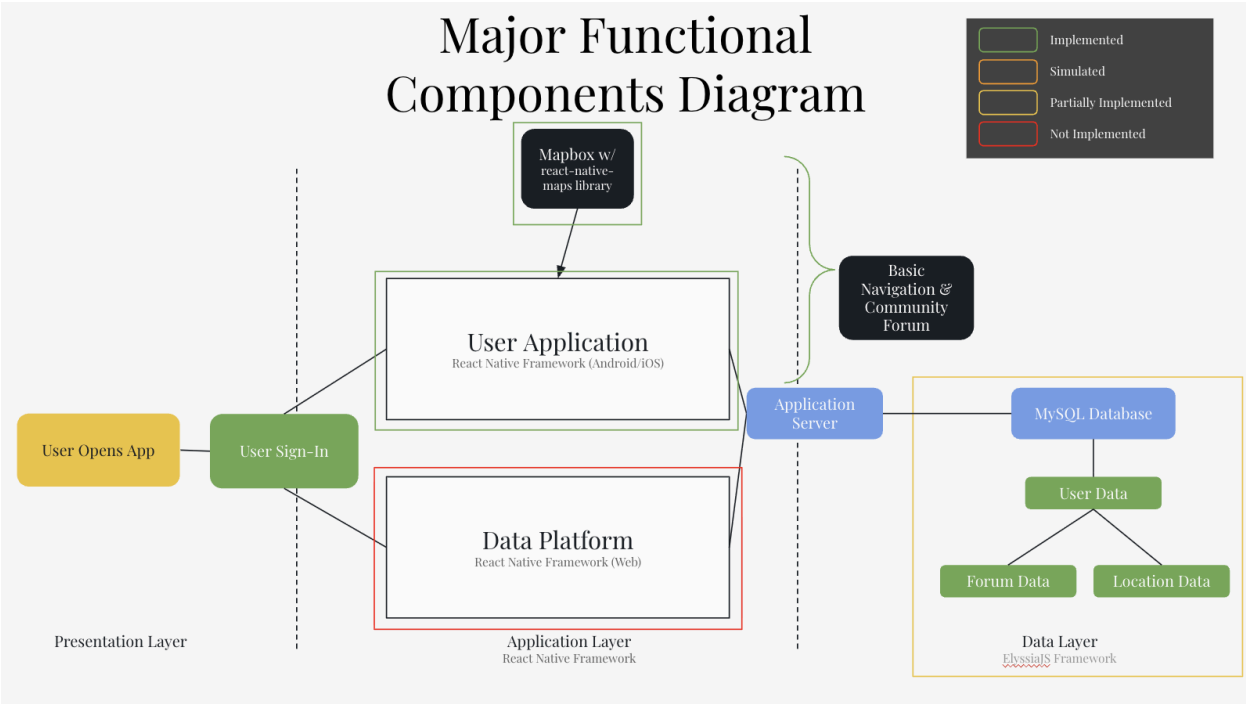

Figure 1. Smart City Major Functional Component Diagram.

2.2 Product Functions

The following outlines the implementation status of the primary system features for the prototype phase:

- Fully Implemented Features:
    - **Login/Authentication**: Secure user validation, registration, and session management.

    - **Account Creation/Deletion**: Secure storage and management of user credentials and profile data.
    - **Access Community Forum**: Full UI and database connectivity for creating, viewing, and interacting with localized posts.
    - **Data Anonymity**: Privacy-by-design implementations protecting user data within the database.
- Partially Implemented Features:
    - **Navigation System**: Scaled down to basic A-to-B navigation rather than the complex Dynamic Queue System envisioned in the RWP.
    - **Alerts and Notifications**: Simulated via mock data rather than utilizing live API push notifications.
    - **Central Information Hub**: Implemented purely as a foundational database for storing basic information, dropping the long-term data platform scalability constraints.
- Eliminated Features:
    - **Add Parking Locations**: Removed due to project time limitations.
    - **Data Analytics and Insights / Visualize Data**: Eliminated, as the analytical dashboard for city officials will not be developed for the prototype.

2.3 User Characteristics

The system is designed to support two distinct user groups:

- Resident / Commuter (Community Members):
    - **Expertise Level**: Minimal technical knowledge. Expected to be familiar with standard smartphone operations and basic GPS navigation apps.
    - **Usage Scenarios**: Community members will use the app daily to look up basic navigation routes. They will actively utilize the community forum to post and read about local event notifications, report immediate safety updates, and share neighborhood traffic conditions to help others avoid congestion.
    - **Constraints**: Constrained by the need for a stable internet connection and an active GPS sensor on their mobile device.

- City Official / Urban Planner:
    - **Expertise Level**: Moderate technical knowledge. Expected to be familiar with viewing database metrics and reading municipal dashboards.
    - **Usage Scenarios**: While the advanced visualization dashboard is eliminated from the prototype scope, in the envisioned system, city officials would use the central information hub to view aggregated, anonymized user location and forum data. They would use this data to identify persistent traffic bottlenecks, monitor community safety concerns, and make data-driven decisions regarding future road infrastructure and resource management.
    - **Constraints**: Must adhere strictly to data anonymity protocols (Privacy-by-Design) to ensure individual users cannot be tracked.

2.4 Constraints

- **Technical Constraints**: The database and server environments must be strictly hosted within the ODU Computer Science department's Virtual Machine. Mobile development is restricted to the capabilities of JavaScript/TypeScript using React Native and ElysiaJS.
- **Time Constraints**: The limited 5-sprint development window inherently restricts the complexity of routing algorithms, requiring a shift from dynamic real-time queuing to static basic routing.
- **Privacy Constraints**: The database schema must be designed to ensure strict data anonymization and user privacy protection when handling location data and account credentials.

2.5 Assumptions and Dependencies

- **Dependencies**: The application is heavily dependent on third-party geospatial services, specifically the Mapbox API and OpenStreetMap API for route rendering, map styles, and geocoding.
- **Assumptions**: It is assumed that the end-user will operate an internet-connected iOS or Android mobile device. Furthermore, it is assumed that the ODU VM infrastructure will maintain continuous uptime to support backend testing and database connectivity during development.